

1. TYPE THIS FIRST (~2 MIN)

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define all(v) (v).begin(),(v).end()

int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    int n; cin >> n;
    vector<ll> a(n);
    for (ll &x : a) cin >> x;

    cout << ans << '\n';
}
```

Debug must go to cerr, never cout. cout debug lands in the answer the judge reads ⇒ silent Wrong Answer that looks like an algorithm bug.

cerr << "dbg " << x << '\n';

2. WHAT FITS IN THE TIME LIMIT

Assume 10^8 ops/sec. Read n , find the row, pick the algorithm **before** writing code.

max n	allowed
≤ 10	$O(n!)$ permutations
≤ 20	$O(2^n n)$ bitmask / subsets
≤ 100	$O(n^3)$ Floyd–Warshall
$\leq 1\,000$	$O(n^2)$ pairwise, 2D DP
$\leq 10^5$	$O(n \log n)$ sort/set/bsearch
$\leq 10^6$	$O(n)$ one pass, prefix sums
$\leq 10^9$	$O(\log n)$, $O(\sqrt{n})$
$\geq 10^{18}$	$O(1)$ formula only

$n \leq 10^5$ **almost always** means: sort it, or put it in a set/map, or binary search it.

3. READ THE CONSTRAINT BACKWARDS

$n \leq 20 \rightarrow$ subset/permutation search. Stop hunting for polynomial.

"min x such that" / "max x such that" $\rightarrow$ **binary search the answer**; you only need a feasibility check.

Huge coordinates, few points $\rightarrow$ coordinate compression.

Sum of n bounded over cases $\rightarrow$ per-case must be near-linear.

$n \leq 5\,000$, 2D feel $\rightarrow O(n^2)$ DP.

4. OVERFLOW

int	$\approx 2.1 \times 10^9$
long long	$\approx 9.2 \times 10^{18}$
double	exact ints to $2^{53} \approx 9 \times 10^{15}$

The #1 cause of WA at this level. $n(n-1)/2$ overflows int at $n \approx 65\,536$ — far below a typical $n \leq 10^5$ bound. Any intermediate past 2×10^9 : long long. Write 1LL << k, not 1 << k.

5. CONSTANT FACTORS THAT CAUSE TLE

cin/cout without sync_with_stdio(false) — up to $10\times$

endl instead of '\n' — flushes every line

map/set — $5\text{--}10\times$ slower than sorting a vector

unordered_map — $O(n)$ worst case under anti-hash tests

v.erase(v.begin()) in a loop — accidental $O(n^2)$

passing containers by value — add const&

6. STL YOU WILL ACTUALLY USE

sort(all(v))	$O(n \log n)$
sort(all(v),greater<>())	descending
reverse(all(v))	
v.erase(unique(all(v)),v.end())	dedupe sorted
lower_bound(all(v),x)-v.begin()	first $\geq x$
upper_bound(all(v),x)-v.begin()	first $> x$
accumulate(all(v),0LL)	sum
max_element(all(v))	iterator
count(all(v),x)	
__gcd(a,b)	
to_string(x) / stoll(s)	
s.substr(i,len)	
priority_queue<T,vector<T>,greater<>> = min-heap.	
for (auto &[k,v] : mp) iterates a map in key order.	

7. PROBLEM SHAPE $\rightarrow$ TECHNIQUE

The problem says	Try
shortest path, all edges cost 1	BFS
shortest path, weights	Dijkstra
"are these connected?", grouping	DSU / BFS
"min/max x such that ..."	binary search answer
pick items, maximise value	DP knapsack
count the ways	DP, watch overflow
"sum of range", many queries	prefix sums
range query + updates	Fenwick / segtree
best contiguous block	Kadane / two pointers
order with dependencies	topological sort
pair things up optimally	sort + greedy
$n \leq 20$, try everything	bitmask
points, area, hull	geometry
anagram / substring / match	hashing, KMP
divisibility, remainders	gcd/lcm, modular
game, who wins	think parity / small cases

If **nothing fits**: write a brute force for $n \leq 8$, print the answers, and look for the pattern. Several NCPC "easy" problems are pure observation with no named algorithm at all.

8. SNIPPETS WORTH KNOWING COLD

```
// BFS, unweighted shortest path
vector<int> d(n, -1); queue<int> q;
d[s] = 0; q.push(s);
while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : adj[u]) if (d[v] < 0) {
        d[v] = d[u] + 1; q.push(v);
    }
}

// binary search the answer
ll lo = 0, hi = 2e18, best = -1;
while (lo <= hi) {
    ll mid = lo + (hi - lo) / 2;    // not (lo+hi)/2
    if (ok(mid)) best = mid, hi = mid - 1;
    else lo = mid + 1;
}

// DSU
int f(int x){ while(p[x]!=x) x=p[x]=p[p[x]]; return x; }
void uni(int a,int b){ a=f(a); b=f(b); if(a!=b) p[a]=b; }

// prefix sums, inclusive [l,r]
vector<ll> P(n+1,0);
for (int i=0;i<n;i++) P[i+1]=P[i]+a[i];
ll s = P[r+1] - P[l];

// grid directions
int dr[]={-1,1,0,0}, dc[]={0,0,-1,1};
```

WRONG ANSWER — IN THIS ORDER

1. **Re-read the output format.** Singular vs plural, capitalisation, Case #1:, trailing text.
2. **Overflow?** Any intermediate $> 2 \times 10^9 \rightarrow$ long long.
3. $n = 0$, $n = 1$, **empty input?** Single-element cases break more solutions than anything else.
4. **Off-by-one.** 0- vs 1-indexed, < vs <=, inclusive ranges.
5. **State reset between test cases?** Clear every global.
6. **Floating point.** cout << fixed << setprecision(10). Never == on doubles. Prefer integers.
7. **Did you misread the problem?** *The most common cause at this level.* Have your teammate read it independently and say the task back to you.
8. **Ties / multiple valid answers?** Check for a tie-break rule.
9. **Do the sample by hand on paper.** If your manual answer differs from the expected output, you misread the problem.

TIME LIMIT EXCEEDED

1. sync_with_stdio(false); cin.tie(nullptr); present?
2. endl $\rightarrow$ '\n'
3. Recompute complexity against the *real* constraints
4. Accidental $O(n^2)$: erase-in-loop, string +=, by-value args
5. unordered_map $\rightarrow$ map or sorted vector
6. Infinite loop / missing visited check

RUNTIME ERROR

1. **Out of bounds** — by far the most common
2. Stack overflow from recursion ($\approx 10^5$ frames) $\rightarrow$ iterative
3. Division / modulo by zero
4. .front(), .top(), .back() on empty container
5. Reading past end of input

BEFORE EVERY SUBMISSION

- ☐ Compiles with no warnings
- ☐ Passes **every** sample, byte for byte

- ☐ Tested $n = 1$ and smallest legal input
- ☐ Everything past 2×10^9 is long long
- ☐ No debug output on cout
- ☐ '\n', not endl

TWO PEOPLE, ONE KEYBOARD

Coder writes the current solution.

Reader does not watch the screen — works the next problem on paper: edge cases, approach, complexity check. Hands over a written plan, then swap.

On WA: the **reader** runs this checklist aloud while the **coder** re-reads the statement. Two people, two different inputs.

THE DAY

- 10:00 doors; log in, open scoreboard
- 11:00 **read all 11. Write nothing.**
A reads A $\rightarrow$ K, B reads K $\rightarrow$ A
mark each E / M / H
- 11:20 start the **shortest** E, not the nicest
- 11:30+ check scoreboard every 20 min
- 12:00 target 1 solved
- 13:00 target 2–3
- 14:00 target 3–4
- 15:00 target 4–5; start nothing new
- 15:45 last realistic submit

The problems are NOT sorted by difficulty — stated verbatim in every NCPC booklet. In 2025 the two hardest (0.4% solved) were F and H, mid-alphabet.

The scoreboard is a free difficulty oracle. With 230+ teams the easy problems light up within 15–20 min. If 40 teams solved G and you marked it H, re-read G — you misjudged it.

REALITY CHECK

Median NCPC team solves **3–5 of 11**. 15–25% of teams solve 0 or 1. Every year 1–2 problems are solved by **zero or one** team out of 230+. You are not expected to solve the hard half.

Stuck 45 min with no AC? Switch problems. Come back later.

Never leave a problem unsubmitted at the end out of fear — a failed submission at 15:55 on a problem you never solve costs **nothing**.